

Flight Price Monitoring Agent

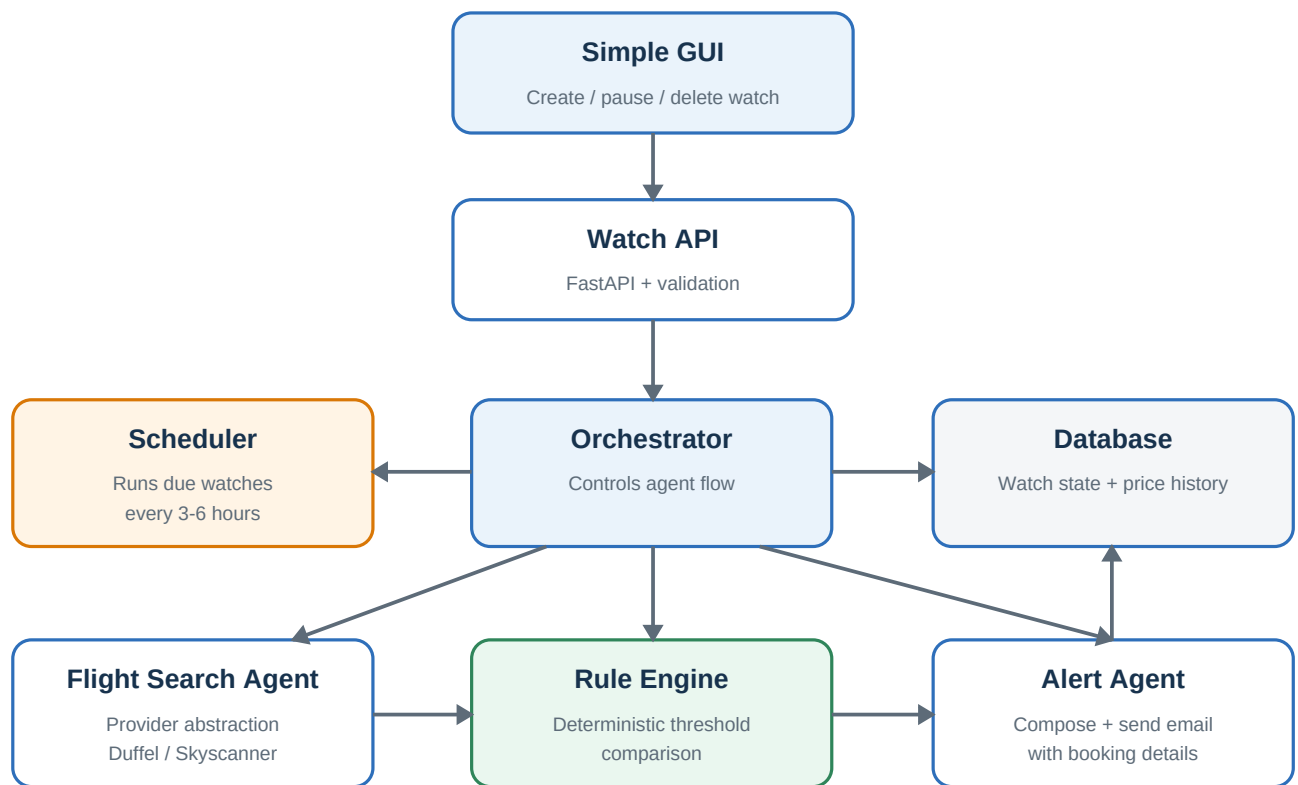
Architecture, product-development plan, checkpoints, and copy-paste prompts for a 5-day MVP build

Product objective. Allow a user to define origin, destination, travel date(s), price threshold, monitoring duration, and email. The service periodically checks live flight offers and sends an alert when a matching fare falls below the threshold.

Recommended MVP stack

Layer	Choice	Why
UI	Streamlit	Fastest path to a usable internal GUI.
API	FastAPI	Simple typed REST endpoints and easy Python integration.
Agent orchestration	LangGraph or small custom state machine	Explicit flow, retries, and tool boundaries without unnecessary autonomy.
Flight provider	Provider interface; start with Duffel, add Skyscanner if access is available	Avoid vendor lock-in and keep provider changes isolated.
Persistence	PostgreSQL	Reliable watch state and price-history storage.
Scheduling	APScheduler locally; AWS EventBridge in cloud	Deterministic periodic execution.
Email	AWS SES or Resend	Reliable transactional email.
Cloud	AWS Lambda + EventBridge + RDS/DynamoDB, or Render/Railway for MVP	Low operational overhead.

1. System architecture



Design principle: agents reason about workflow and presentation; deterministic code owns prices, thresholds, dates, scheduling, persistence, deduplication, and expiry.

Core components

- **Watch UI:** collects route, date, threshold, duration, filters, and recipient email.
- **Watch API:** validates requests and creates/updates/deletes watches.
- **Scheduler:** selects watches whose `next_check_at` is due and whose `expires_at` has not passed.
- **Flight Search Agent:** calls one or more provider adapters and converts results into a common internal offer format.
- **Rule Engine:** applies numeric rules such as `price <= threshold`, optional stop count, airline, and time-window filters.
- **Alert Agent:** summarizes the qualifying offer and generates a concise email containing fare, itinerary, provider, and booking URL.
- **Database:** remains the source of truth for watch state, execution state, alert history, and price history.

Minimal data model

Table	Important fields
flight_watch	id, origin, destination, departure_date, return_date, threshold, currency, email, interval, expires_at, next_check_at, status
price_history	watch_id, checked_at, lowest_price, airline, flight_number, stops, provider, booking_url
alert_history	watch_id, offer_fingerprint, price, sent_at, delivery_status

DAY 1	Flight data integration
Goal	Prove that the system can retrieve and normalize real flight offers reliably.

Task	Deliverable / Checkpoint
Create repository, Python environment, configuration, and secrets handling.	Application starts locally; API key never appears in source control.
Implement a provider interface such as <code>search_flights(request)</code> .	Provider-specific code is isolated behind one interface.
Integrate the first flight provider and normalize results.	At least the cheapest 5 offers contain fare, airline, schedule, stops, provider, and booking reference/URL when supplied.
Add basic retries, timeout handling, logging, and mocked tests.	Invalid routes, empty results, provider errors, and timeouts fail cleanly.

Day 1 checkpoint

Given a route and date, a local command or API request returns normalized offers sorted by total price. Do not move on until this is reliable.

Prompt for your coding agent

You are the senior backend engineer for a flight-price monitoring product. Build Day 1 only. Use Python and FastAPI. Create a provider abstraction called `FlightProvider` with a `search_flights()` method. Implement one concrete provider adapter for the flight API configured through environment variables. Normalize every provider response into our internal `FlightOffer` model with: origin, destination, departure datetime, arrival datetime, airline, flight number, stops, total price, currency, provider name, offer id, and booking URL when available. Add request validation, structured logging, provider timeouts, bounded retries for transient failures, and unit tests using mocked API responses. Never hard-code secrets. Return the 5 cheapest normalized offers sorted by total price. Keep the architecture small and production-oriented. At the end, provide exact commands to run the service and tests locally.

DAY 2	Persistent monitoring engine
Goal	Turn a one-time search into a stateful watch that can survive restarts.

Task	Deliverable / Checkpoint
Create FlightWatch and PriceHistory persistence models.	Create/read/update/pause/delete watch operations work.
Implement due-watch selection using next_check_at, expires_at, and status.	A restart does not lose monitoring state or duplicate scheduling.
Run scheduled searches and persist the cheapest offer for each execution.	Price history shows timestamped checks for each watch.
Add idempotent run identifiers and basic failure state.	Repeated scheduler invocation does not create duplicate execution records.

Day 2 checkpoint

Create a seven-day watch, run the worker multiple times, and verify that only due watches execute and price history is retained.

Prompt for your coding agent

Continue the existing flight-price project with Day 2. Add PostgreSQL persistence using SQLAlchemy and migrations. Create FlightWatch, PriceHistory, and ExecutionRun models. FlightWatch must include origin, destination, departure_date, optional return_date, price_threshold, currency, email, check_interval_minutes, created_at, expires_at, next_check_at, and status. Implement REST endpoints to create, list, pause, resume, and delete a watch. Add a monitoring worker that selects only ACTIVE watches where next_check_at <= now and expires_at > now, calls the existing FlightProvider, saves the cheapest normalized offer, and advances next_check_at. Make scheduler executions idempotent so retries do not create duplicate runs. Add tests for expiry, pause/resume, no-results, provider failure, and restart-safe scheduling. Do not add agents or email yet.

DAY 3	Agentic orchestration and alerts
Goal	Introduce controlled agents around the deterministic monitoring core and deliver alert emails.

Task	Deliverable / Checkpoint
Create an orchestrator graph/state machine around the monitoring run.	State transitions are visible and testable.
Implement Search Agent as a tool-using wrapper over provider adapters.	Agent can choose/fallback between configured providers without changing rule logic.
Implement deterministic threshold and filter rules.	Qualifying decision is made by code, not an LLM.
Implement Alert Agent and email service.	A qualifying offer sends exactly one useful email; duplicate alerts are suppressed.

Recommended flow

Load Watch -> Search Agent -> Normalize -> Persist -> Rule Engine -> [No Match: End] / [Match: Alert Agent -> Email -> Record Alert]

Day 3 checkpoint

Use a deliberately high threshold to force an alert, then a deliberately low threshold to verify that no alert is produced. Repeat the same qualifying offer and confirm deduplication.

Prompt for your coding agent

Continue the project with Day 3 and make the workflow agentic without moving deterministic business rules into the LLM. Use LangGraph if appropriate; otherwise implement an explicit typed state machine. Create three logical nodes: Search Agent, Rule Engine, and Alert Agent. The Search Agent may select among configured FlightProvider adapters and return normalized offers. The Rule Engine must be plain deterministic Python and evaluate price <= threshold plus optional stops, airline, and time-window filters. The Alert Agent may use an LLM only to compose a concise human-readable message from validated structured offer data; it must never invent fare values, schedules, airlines, or URLs. Add an EmailService abstraction with an SES or Resend implementation. Store alert history and generate an offer fingerprint so the same qualifying offer is not repeatedly emailed unless the price improves by a configurable amount. Add tests for match, no-match, duplicate suppression, email failure, fallback provider, and malformed LLM output.

DAY 4	Simple GUI and product usability
Goal	Make the system usable without touching code or the database.

Task	Deliverable / Checkpoint
Build Streamlit form for route, date(s), threshold, duration, interval, filters, and email.	A non-developer can create a valid watch.
Add active-watch list with status, last check, lowest seen price, and next check.	User can understand current monitoring state at a glance.
Add pause/resume/delete actions.	Lifecycle can be managed completely from the GUI.
Add basic price-history chart and error/status messages.	Recent price movement and failures are visible.

Day 4 checkpoint

Create, pause, resume, and delete a watch from the GUI. Confirm the GUI does not directly call the flight provider; it talks only to your backend API.

Prompt for your coding agent

Continue the project with Day 4. Build a minimal Streamlit UI that consumes the existing FastAPI endpoints and does not access the database or flight provider directly. The create-watch form needs origin airport, destination airport, departure date, optional return date, target price, currency, monitoring duration in days, check interval, email address, maximum stops, optional airline filter, and optional departure-time window. Add an Active Watches section showing route, travel date, threshold, current status, last checked time, current/lowest observed price, next scheduled check, and buttons for pause, resume, and delete. Add a simple price-history line chart for a selected watch. Validate user input clearly and display backend failures without exposing stack traces or secrets. Keep the layout compact and functional rather than decorative.

DAY 5	Cloud deployment and one-week pilot
Goal	Deploy the full system, add observability, and start a real seven-day monitoring run.

Task	Deliverable / Checkpoint
Containerize and deploy API/UI/worker or map them to serverless components.	Publicly reachable UI/API with secrets stored securely.
Configure cloud scheduler and database backups/retention.	Due watches execute without keeping a developer laptop online.
Add health checks, structured logs, execution metrics, and alert failure logging.	A failed search or email can be diagnosed quickly.
Run an end-to-end pilot with a real watch for seven days.	All executions, prices, alerts, and failures can be audited from stored state/logs.

Day 5 checkpoint

The service survives restarts, executes schedules in the intended timezone, does not leak secrets, and can send a verified alert from cloud infrastructure.

Prompt for your coding agent

Continue the project with Day 5. Prepare a production-like MVP deployment. Prefer AWS serverless where practical: API Gateway/Lambda for the API, EventBridge Scheduler for periodic executions, SES for email, and a managed persistent database appropriate to the existing SQLAlchemy design; otherwise provide a Docker-based Render/Railway deployment path. Store credentials in a secret manager or environment configuration, never in code. Add health/readiness endpoints, structured logs with watch_id and execution_run_id, counters for searches/matches/provider failures/email failures, and safe error handling. Make all timestamps UTC internally while accepting/displaying the user's configured timezone. Add deployment configuration, environment-variable documentation, database migration instructions, and an end-to-end smoke test. Finally provide a pilot checklist for running one real watch for seven days and verifying every scheduled execution.

2. Product checkpoints and acceptance criteria

Area	MVP acceptance criterion
Data quality	Normalized price is the provider-reported total used for threshold comparison; currency is explicit.
Reliability	Transient API errors retry with limits; permanent failures are recorded and do not crash other watches.
State	All watches have explicit ACTIVE/PAUSED/EXPIRED state plus next_check_at and expires_at.
Alerting	A qualifying fare sends one alert with route, date/time, airline, fare, stops, provider, and booking details available from the provider.
Deduplication	Same offer is not repeatedly emailed unless the configured re-alert condition is met.
Security	API keys and email credentials are never committed or returned to the browser.
Observability	Every execution can be traced by watch_id and execution_run_id.
Usability	Create/pause/resume/delete is possible through the GUI.
Cloud readiness	A developer machine can be switched off without stopping monitoring.

Suggested V2 backlog

- Multi-date or flexible-date searches and round-trip optimization.
- Percentage-drop alerts, re-alert thresholds, and price-trend summaries.
- Provider fan-out with rate limiting and best-offer reconciliation.
- Non-stop/airline/baggage/refundability filters.
- User accounts, multiple recipients, and notification channels such as Telegram or push.
- Cost controls, provider quotas, caching, and adaptive search frequency as departure approaches.

Build discipline

Treat the five days as vertical slices. At the end of every day, keep the repository runnable, tests green, and the previous day's capability intact. The most important engineering choice is the provider abstraction: every external flight API should be replaceable without touching scheduling, rule evaluation, persistence, or alerting.